

Round Robin Algorithm

#include <stdio.h>

int main()

{

int n, tq;

printf("Enter number of processes:");

scanf("%d", &n);

int id[n], at[n], bt[n], rt[n];

int ct[n], tat[n], wt[n], start[n];

for (int i = 0; i < n; i++)

id[i] = i + 1;

printf("\n Arrival time for P%d: ", i + 1);

scanf("%d", &at[i]);

printf("Burst Time for P%d: ", i + 1);

scanf("%d", &bt[i]);

rt[i] = bt[i];

start[i] = -1;

}

printf("\n Enter Time Quantum: ");

scanf("%d", &tq);

for (int i = 0; i < n - 1; i++) // Sort processes by arrival time

for (int j = 0; j < n - i - 1; j++)

{

if (at[j] > at[j + 1])

{

int temp;

temp = at[j]; at[j] = at[j + 1]; at[j + 1] = temp;

temp = bt[j]; bt[j] = bt[j + 1]; bt[j + 1] = temp;

temp = rt[j]; rt[j] = rt[j + 1]; rt[j + 1] = temp;

temp = id[j]; id[j] = id[j + 1]; id[j + 1] = temp;

}

}


```

int queue[100], front=0, rear=0;
int visited[n];

for (int i=0; i<n; i++) visited[i]=0;

int time = at[0];
queue[rear++] = 0;
visited[0] = 1;

int ganttP[200], ganttT[200], g=0;
float sumTAT=0, sumWT=0, sumRT=0;

printf("\n --- Round Robin Scheduling ---");

while (front < rear)
{
    int i = queue[front++];
    if (start[i] == -1)
        start[i] = time;

    ganttP[g] = id[i];
    ganttT[g] = time;
    g++;

    if (rt[i] > tg)
    {
        time += tg;
        rt[i] -= tg;
    }
    else
    {
        time += rt[i];
        rt[i] = 0;
        ct[i] = time;
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        sumTAT += tat[i];
        sumWT += wt[i];
        sumRT += (start[i] - at[i]);
    }
}

```



```

for (int j = 0; j < n; j++) {
    if (at[j] <= time && visited[j] == 0) {

```

```

        queue[rear++] = j;

```

```

        visited[j] = 1;

```

```

    }
    if (rear > 0) {
        queue[rear++] = i;

```

```

    if (front == rear) {

```

```

        for (int j = 0; j < n; j++) {

```

```

            if (visited[j] == 0) {

```

```

                time = at[j];

```

```

                queue[rear++] = j;

```

```

                visited[j] = 1;

```

```

                break;

```

```

            }

```

```

        }

```

```

    }

```

```

    ganttT[g] = time;

```

```

    printf("In Gantt Chart: \n");

```

```

    for (int i = 0; i < g; i++) {

```

```

        printf("P %d ", ganttT[i]);

```

```

        printf("In %d", ganttT[i]);

```

```

        for (int i = 1; i <= g; i++) {

```

```

            printf("%d", ganttT[i]);

```



```

printf("\n\n Process | EAT | BT | CT | TAT | WT | RT |",
for (int i = 0; i < n; i++)
{
    printf("\n P%d | %d | %d | %d | %d | %d | %d |",
           id[i], at[i], bt[i], ct[i], tat[i], wt[i], start[i] - at[i]);
}

printf("\n\n Average SAT : %.2f", sum TAT/n);
printf("\n\n Average WT : %.2f", sum WT/n);
printf("\n\n Average RT : %.2f\n", sum RT/n);
return 0;
}

```

Q.P/

Enter number of processes: 6

Arrival Time for P1: 5

Burst Time for P1: 5

Arrival Time for P2: 4

Burst Time for P2: 6

Arrival Time for P3: 3

Burst Time for P3: 7

Arrival Time for P4: 1

Burst Time for P4: 9

Arrival Time for P5: 2

Burst Time for P5: 2

Arrival Time for P6: 6

Burst Time for P6: 3

Enter Time Quantum: 4

Round Robin Scheduling

Gantt Chart:

P4 | P5 | P3 | P2 | P1 | P4 | P6 | P3 | P2 | P1 | P4 |

1 5 7 11 15 19 23 26 29 31 32 33

Process	AT	BT	CT	TAT	WT	RT
P4	1	9	33	32	23	0
P5	2	2	7	5	3	3
P3	3	7	26	19	14	14
P2	4	6	31	27	21	7
P1	5	5	32	27	22	10
P6	6	3	26	20	17	17

Average TAT: 22.83

Average WT: 17.50

Average RT: 6.83

gk

21/4